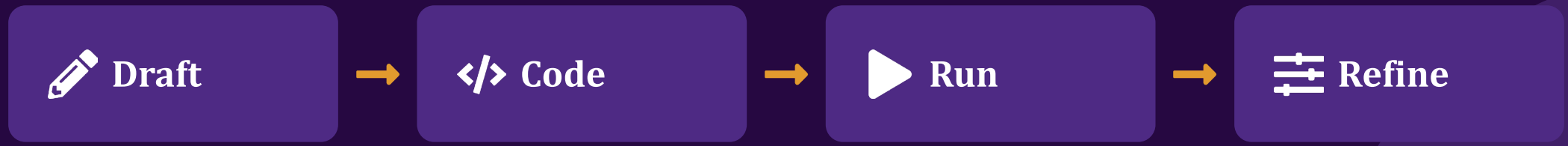
HANDS-ON WORKSHOP · ~2 HOURS

From Prompt to Publication: Data Visualization with LLMs

Rapidly prototype figures by prompting — then turn the rough draft into reproducible, publication-ready code you run yourself.



Presented by Aaron Geller, Lead Data Scientist, Research Computing and Data Services, Northwestern IT
First draft of slides created by Claude (Anthropic), with prompting and input files from Aaron Geller

This workshop is brought to you by:

Northwestern IT
Research Computing and Data Services

Need help?

- AI, Machine Learning, Data Science
- Statistics
- Visualization
- Collecting web data (scraping, APIs), text analysis, extracting information from text
- Cleaning, transforming, reformatting, and wrangling data
- Automating repetitive research tasks
- Research reproducibility and replicability
- Programming, computing, data management, etc.
- R, Python, SQL, MATLAB, Stata, SPSS, SAS, etc.

Request a **FREE** consultation at **bit.ly/rcdsconsult**.

SETUP

Before we start

Pick an LLM

Any chat assistant with file upload + code execution: **Claude, ChatGPT, Gemini, Copilot.**

Free tiers work fine; paid is optional. Your choice today.

Pick a way to run code

Google Colab (nothing to install) or your **local Python**

Hitting setup snags locally? Switch to Colab and keep moving.

We'll demo in Python. But the same *Draft* → *Code* → *Run* → *Refine* steps also apply to R —you're welcome to work in R today.

Datasets are on GitHub : https://github.com/nuitrcs/AI-for-data-visualization-workshop

WHY WE'RE HERE

I went in as a skeptic

> *"When I first heard people were using LLMs to make their data visualizations, my reaction was:* ***don't do that."***

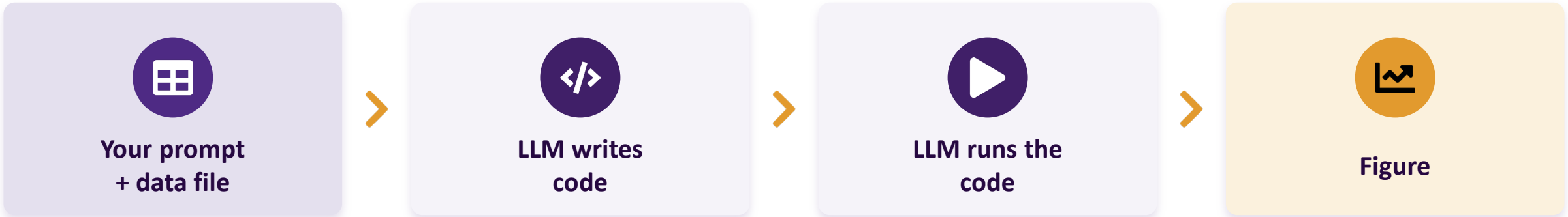
So, I ran a systematic test = same prompts, across many tools and data types. The verdict:

Cautious optimism — with real caveats.

This workshop turns that finding into a workflow you can run yourself.

THE MENTAL MODEL

What's actually happening



The LLM is writing code — and the code can be wrong.

This is also why it works at all: Python (or R) can read almost any data format, analyze it, and visualize it — without you writing a line of code. The LLM is harnessing that flexibility on your behalf.

TODAY'S BIG IDEA

The LLM figure is a rough draft. The code is the deliverable.

1. Draft

Prompt → figure, fast. Something to react to.

2. Code

Ask for the standalone script behind it.

3. Run

Execute it yourself — reproducible & verifiable.

4. Refine

Iterate to publication quality.

ROADMAP

The plan

1. **The Rough Draft** *prompt → figure* — ~20 min
2. **From Draft to Code** *get it out & run it* — ~25 min
3. **Refine to Publication-Ready** *polish & export* — ~20 min
4. **Your Data / Other Formats** *open exploration* — ~20 min

Plus framing & wrap-up — roughly 20 min.

You'll leave able to:

- Prompt any LLM to draft a figure from your data
- Get code out of the chat
- Run & reproduce it (Colab or local)
- Refine your figure to publication quality

PART 1

The Rough Draft

Prompt → figure, in seconds.

~20 min

PART 1

Anatomy of a good first prompt

Describe your data

What is in a given row? What do the columns mean? Mention the file type so the tool knows how to read it.

Describe the desired figure

Describe the relationship or comparison you want. Experiment with specifying the chart type or letting the LLM choose.

Specify the output

Ask for a file you can download, e.g., PDF or PNG (and, in our next part, we'll also ask for the code to generate the figure).

The clearer your prompt, the better. The LLM will make assumptions to fill in any gaps — sometimes correct and sometimes not — and those assumptions are exactly what you'll need to verify later.

PART 1-3 · DATASET

Our dataset: Chicago Public Schools

`CPS_merged.csv`

~650 rows (one per school), 280 columns. A real, slightly messy CSV.

The goal

A scatter plot of mobility rate vs. college enrollment — with a fitted trend line and 1σ confidence band.

A trap to watch for

Several columns have very similar names (e.g., more than one “college enrollment” field). LLMs tend to pick one silently. Note which one yours chooses — we'll come back to this.

*More information on each dataset + example prompts are in the repo's **data/README.md** file.*

EXERCISE 1 ~10 min

Make a rough draft

1. Open your chosen LLM.
2. Upload CPS_merged.csv (the data/ folder in the repo).
3. Paste the starter prompt →
4. Look at what comes back. Did it pick the columns you expected?

STARTER PROMPT

```
I will upload a data file. Each row is a
different school, and each column is a
different attribute of that school. Create a
scatter plot from these data showing the
mobility percentage vs. the college
enrollment percentage. Include a trendline
fit to these data and show the 1 sigma
confidence interval for that line. Provide
the figure in a PDF format that I can
download and include in my research paper.
```

Share with us your draft, and tell us which tool you used.

PART 1

Share

Tell us

- Which tool did you use?
- Did it pick the columns you intended?
- Did it add anything you didn't ask for — an extra chart, an aggregation, a caption?
- Did it hand you the code, or just the picture?

Takeaway

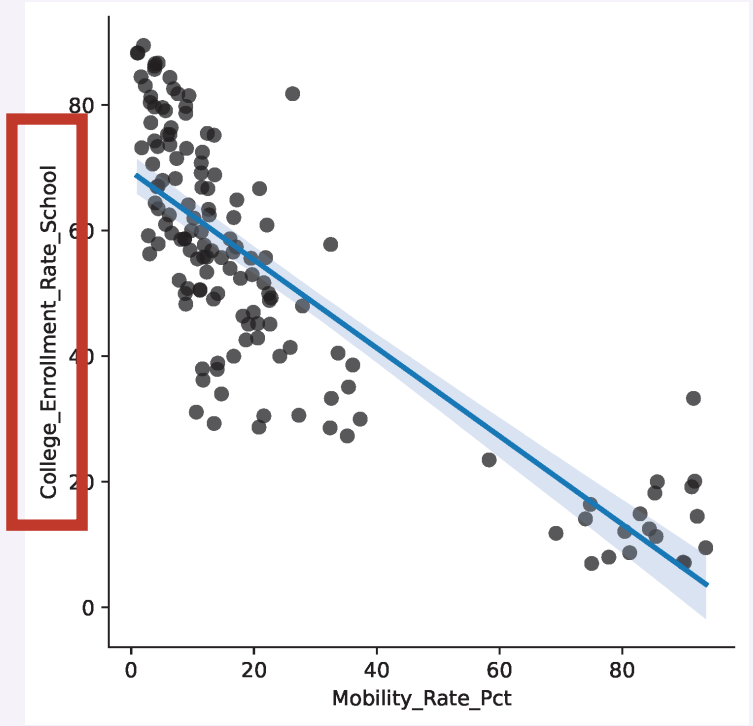
It works — a usable plot from one plain-English prompt.

But every tool will make **silent choices.** That's exactly why we don't stop here.

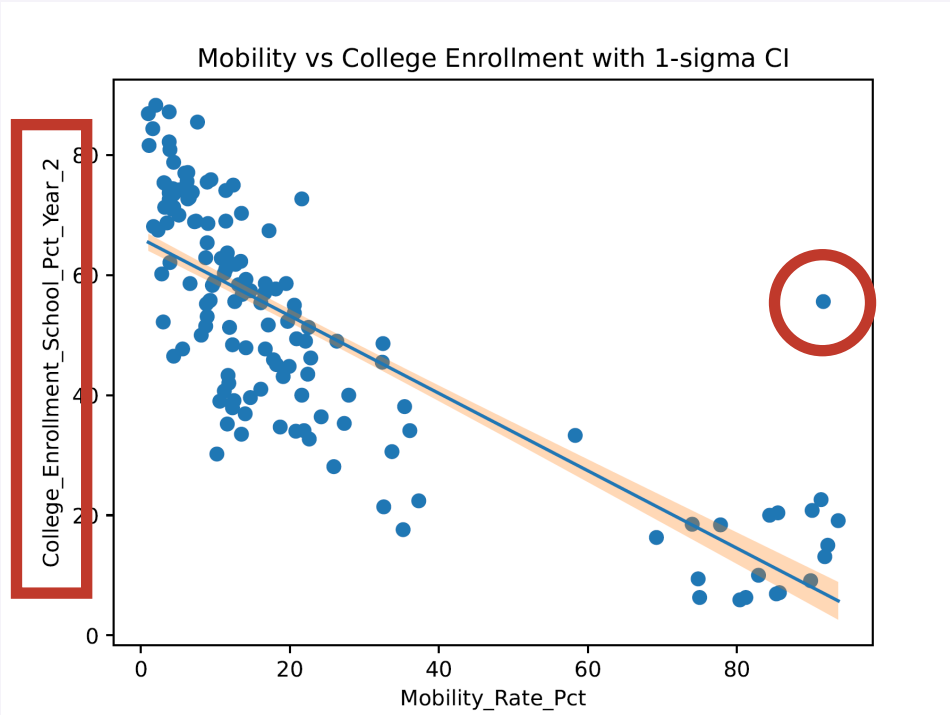
A REAL EXAMPLE FROM MY DRAFTS

What can go wrong

Silent (incorrect) choices



My Figure (produced using Python w/o an LLM)

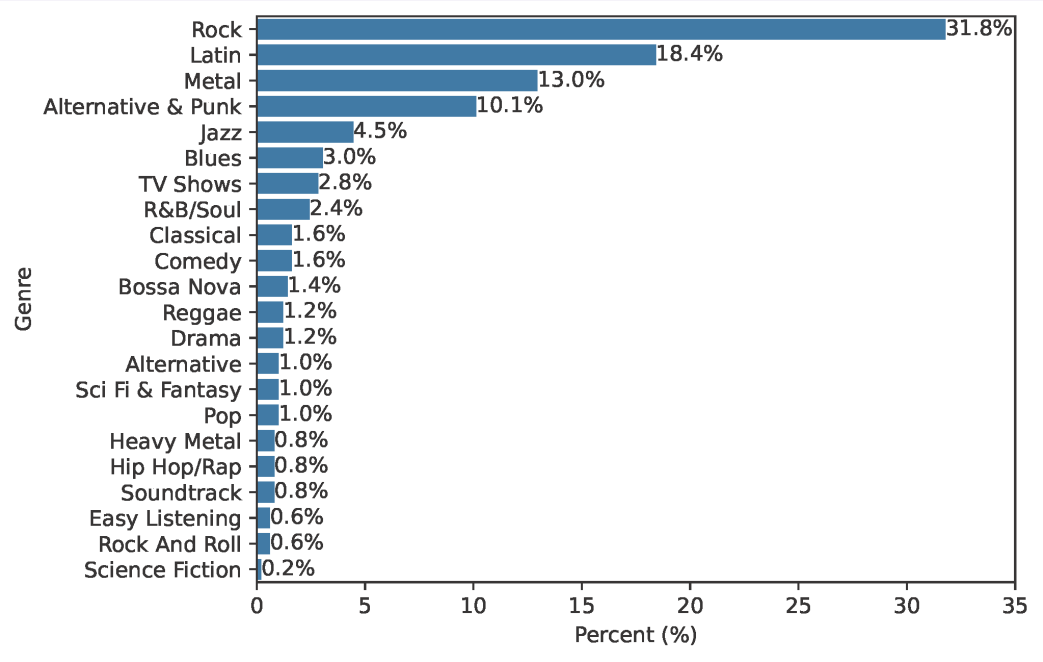


ChatGPT silently used a different "enrollment" column.
(And luckily here it's clear from the figure label…)

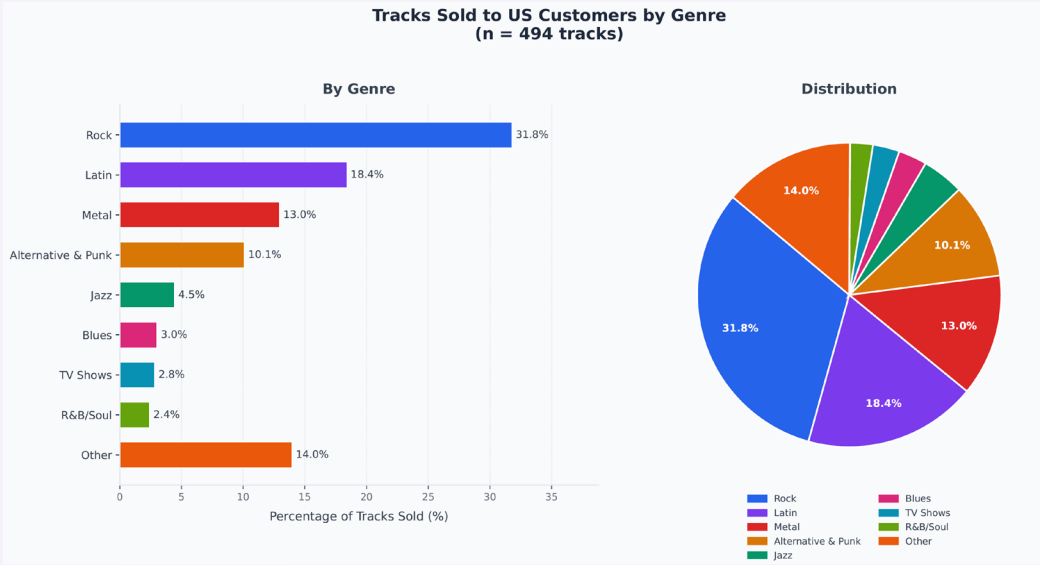
A REAL EXAMPLE FROM MY DRAFTS

What can go wrong

Uninvited decisions



My Figure (produced using Python w/o an LLM)

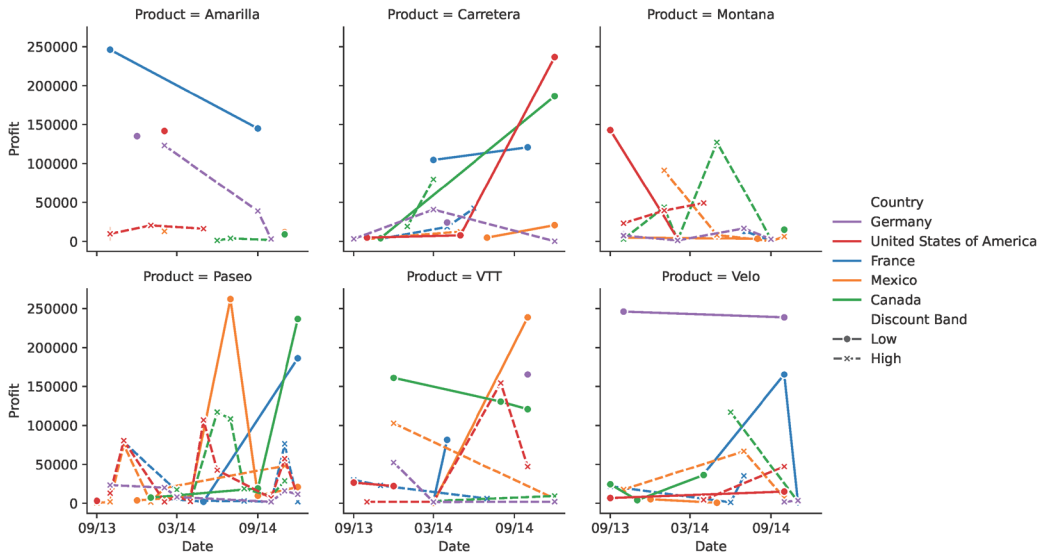


Claude I didn't ask for that pie chart …

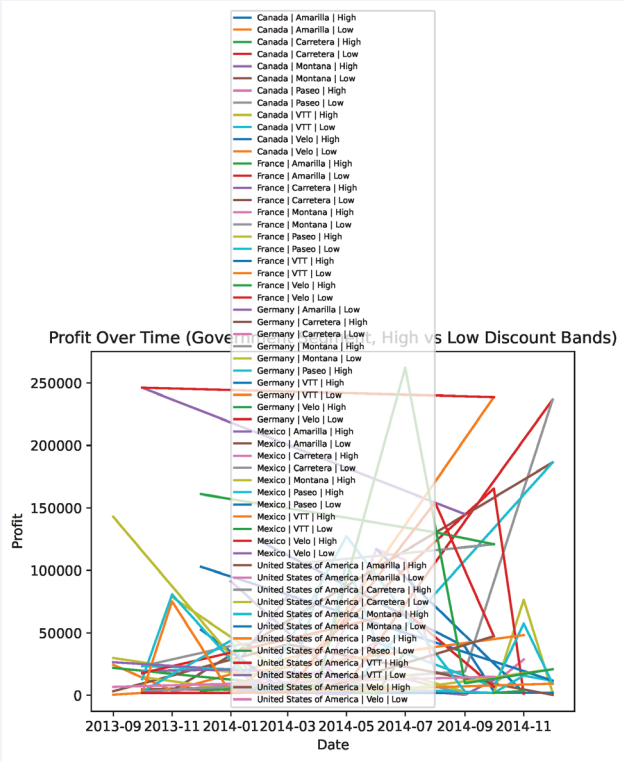
A REAL EXAMPLE FROM MY DRAFTS

What can go wrong

Unusable plots



My Figure (produced using Python w/o an LLM)

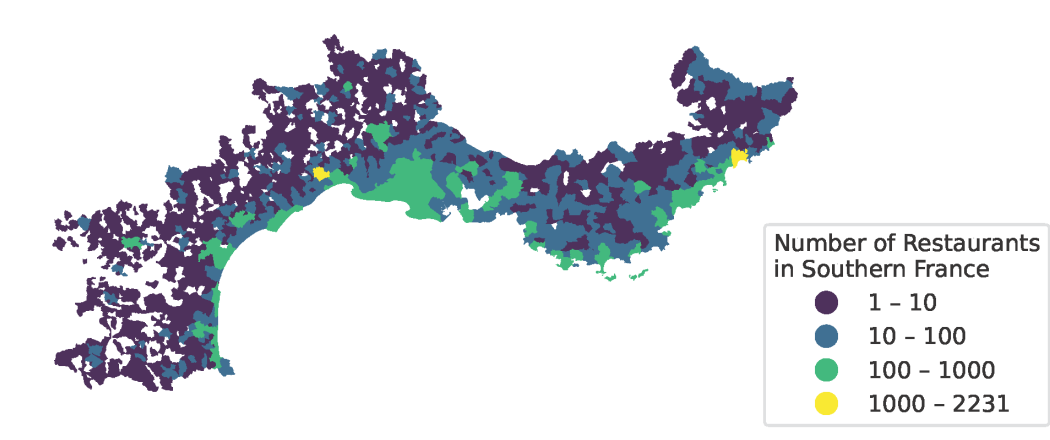


ChatGPT … um … this is surprisingly bad (though I tried again a month later and it produced a much better figure)

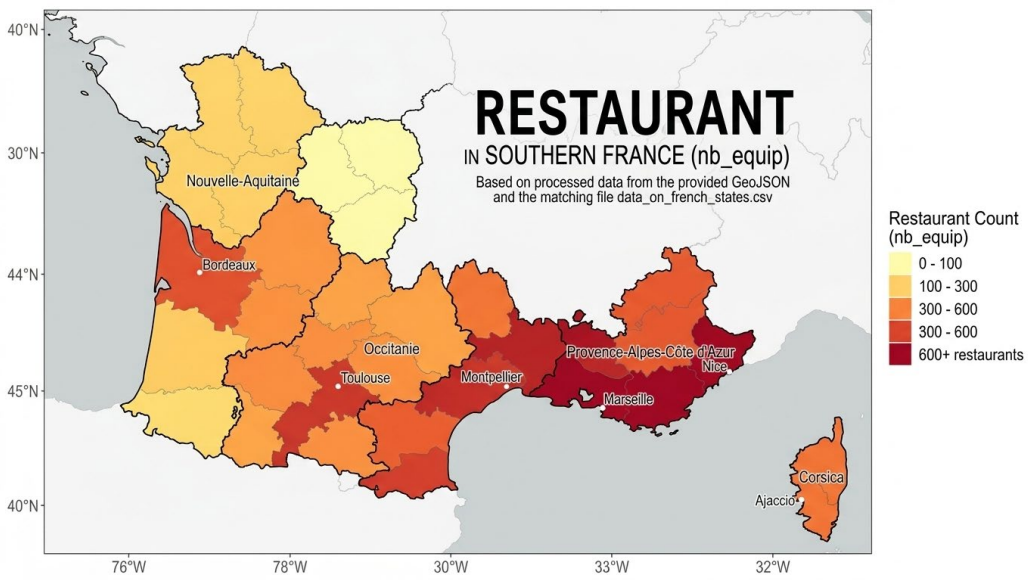
A REAL EXAMPLE FROM MY DRAFTS

What can go wrong

Didn’t share the code



My Figure (produced using Python w/o an LLM)

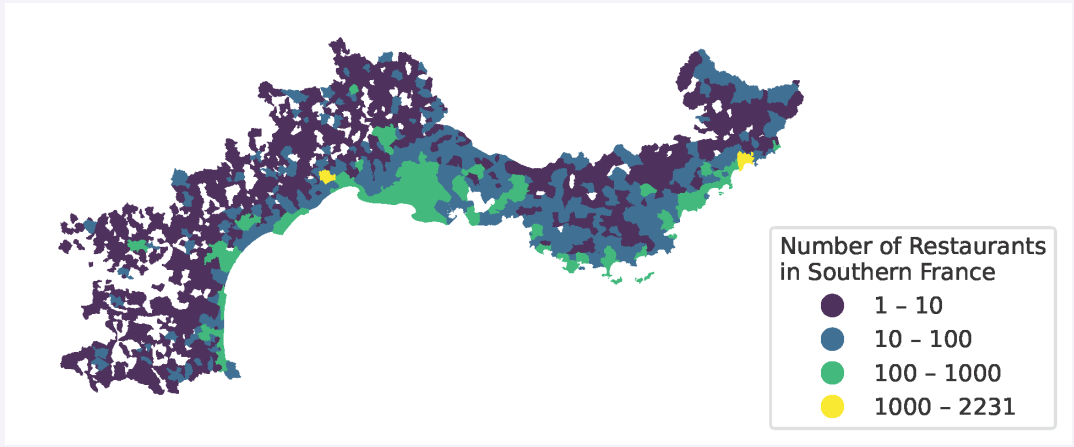


Gemini this is gorgeous, but … it did not share the code initially... so I have no way to verify if it is correct.

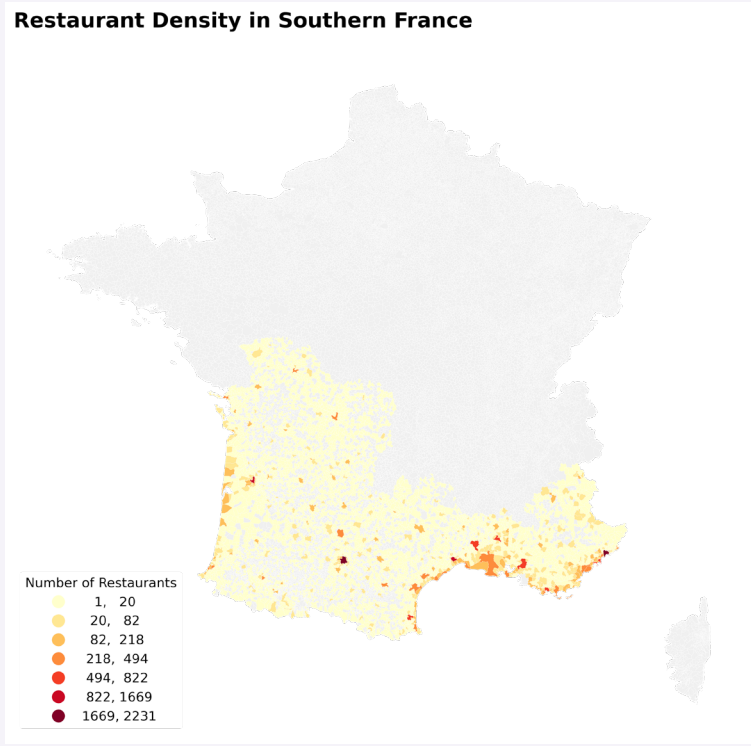
A REAL EXAMPLE FROM MY DRAFTS

What can go wrong

Didn’t share the code



My Figure (produced using Python w/o an LLM)



Gemini When I asked later for the code, this is the figure the code produced…

CHECKLIST

Before you use that LLM figure

1	**Does it show what you asked for?**	Compare the figure to your original prompt.
2	**Did it use the right data?**	Check axis labels and column names. Any unexpected filtering or aggregation?
3	**Does it make intuitive sense?**	Sanity-check: does the trend match what you know about the data?
4	**Read & run the code ★**	The most important step. Re-run it yourself to confirm it's reproducible.

Not 100% certain? Submit a consultation request to our RCDS-DSSV team — we can review the code and help you to catch subtle errors a non-expert might miss.

PART 2

From Draft to Code

The figure is disposable. The code is the asset.

~25 min

PART 2

Get the code — and run it yourself

Why bother?

- **Reproducible** — re-run anytime, hand it to a colleague
- **Verifiable** — read what the LLM actually did with your data to produce the figure
- **Editable** — you control every detail of the final figure
- **Portable** — works beyond the chat's sandbox, limits, and token caps

Use a prompt similar to this:

```
Show me the complete, standalone
Python script that produced this
figure, including how to load the
data. Keep it as short and simple as
you can.
```

Working in R? Same move — just ask for a “standalone R script.” Everything in this round transfers.

PART 2

Getting your data into the code

Local Python

File sits next to your script:

```
pd.read_csv('CPS_merged.csv')
```

Colab

Use the Files panel to upload, or read straight from a URL:

```
pd.read_csv('https://raw.githubusercontent.com/…/CPS_merged.csv')
```

Ask the LLM

Tell it where the data lives and let it write the loading code:

```
The data is at this raw GitHub URL: <url>. Load it directly from there.
```

Use the raw.githubusercontent.com link (GitHub's “Raw” button) — not the github.com page URL, which returns a web page.

LIVE DEMO

I’ll demo the workflow: draft → code → run

1. Ask the LLM for the complete standalone script.
2. Copy it into Colab (or local Python).
3. Point it at the data and run it.
4. The same figure appears — now outside the chat.
5. Review the checklist : what I asked for? right data? right columns? does it make sense?

I’ll demo this live first. Then you can try it yourself

EXERCISE 2 ~10 min

Your turn: get the code out and run it

1. Ask your LLM for the complete standalone script (use the phrasing from the last slide).
2. Open Colab, or your local Python.
3. Choose how you will load the data: upload it, read the raw URL, or let the LLM wire it up.
4. Run your code. Reproduce your figure outside the chat.
5. Review the checklist.

Hit an error? Paste the full error message back to the LLM and ask it to fix it. Repeat as needed.

PART 2

Share

Tell us

- Did your figure reproduce when you ran the code?
- What errors came up — and did pasting them back in the LLM actually fix them?
- How long was the generated code vs. what the task really needs?
- Did anything in the code surprise you about how it handled your data?

Takeaway

If you can read, understand and re-run the code, you can **trust it, share it, and build on it.**

PART 3

Refine to Publication-Ready

From “it runs” to “it ships.”

~20 min

PART 3

Two ways to refine

Re-prompt

Describe the change in plain English and let the LLM rewrite the code. Best for bigger moves and when you're not sure how to code it.

Edit the code yourself

Change colors, labels, or limits by hand. Faster for small, precise tweaks — and you learn the plotting library as you go.

AIM FOR PUBLICATION QUALITY

- clear labels & units
- honest axes
- no chartjunk
- readable fonts
- vector PDF / SVG at the right size

Note: we have a full workshop of data visualization best practices (keep an eye on our schedule)

EXERCISE 3 ~10 min

Polish it

Pick a few changes. Re-prompt or edit the code directly:

- ✓ Change the marker style (e.g., so you can see overlap)
- ✓ Change the line style
- ✓ Drop the legend; remove the top & right spines
- ✓ Improve axis titles, e.g., clearer wording, add units
- ✓ Export as a PDF

Stretch goal

Ask for a short caption and/or set the figure size for a two-column paper.

Share-back: what did you change, was re-prompting or hand-editing faster?

PART 3 · EXTENSION

Small wording, big difference

Same data, same tool — but asking for a figure “for my research paper” vs. “for my presentation” can produce visibly different figures.

“...for my research paper”

Tends toward denser detail, a caption, neutral palette, smaller fonts — built to sit in a column of text.

“...for my presentation”

Tends toward bigger fonts, bolder colors, less text — built to read from across a room.

Tell the tool the context and audience. This should be part of the initial spec, not an afterthought.

PART 4

Your Data, Other Formats

Explore this workflow with a dataset of your choice.

~20 min

PART 4

Pick your dataset

`.xlsx`

Multi-line trends over time

filtering & grouping

`.sqlite / .db`

Bar chart from a query

the LLM writes SQL

`.geojson + .csv`

Choropleth map

join two files; watch file size

`.vtk`

3D volume rendering

ambitious — verify carefully

`Your own data`

Whatever you need

the real test

*More information on each dataset + example prompts are in the repo's **data/README.md** file.*

EXERCISE 4

~10 min + share

Run the workflow on a new format

1. Pick a dataset; yours or from our repo
2. Draft with LLM → ask for the code → run it on your own.
3. Note where this worked and where it struggled

Share

Wins and instructive failures are both welcome.

Some file formats are easier than others: big files (e.g., the .geojson) may exceed upload limits, and 3D rendering (e.g., the .vtk) often needs a follow-up prompt. That friction is part of this exercise.

CHECKLIST

Before you use that LLM figure

1	**Does it show what you asked for?**	Compare the figure to your original prompt.
2	**Did it use the right data?**	Check axis labels and column names. Any unexpected filtering or aggregation?
3	**Does it make intuitive sense?**	Sanity-check: does the trend match what you know about the data?
4	**Read & run the code ★**	The most important step. Re-run it yourself to confirm it's reproducible.

Not 100% certain? Submit a consultation request to our RCDS-DSSV team — we can review the code and help you to catch subtle errors a non-expert might miss.

WRAP-UP

Always verify

Good fit for LLM drafting

- ✓ Exploratory plots while you plan
- ✓ Standard chart types (scatter, bar, line)
- ✓ Common, well-supported file formats
- ✓ Code you can read and re-run yourself

Slow down / get help

- Figures feeding decisions or for publication
- Unusual formats or very large files
- Anything you can't verify yourself

CONCLUSION

“The floor just got higher — but the ceiling still requires expertise.” (Claude)

Try it with your own data

Run the workflow on your own data — draft, get the code, run it.

Failed before? Retry

These tools change fast. Last month's “no” is often this month's “yes.”